Rapid prototyping and verification of cyber-physical security controls for SIDCs using bigraphs and model checking

Mattia Bradascio, Gabriel Kulp, Jakub Krýs

Frontier AI Security Residency 2026

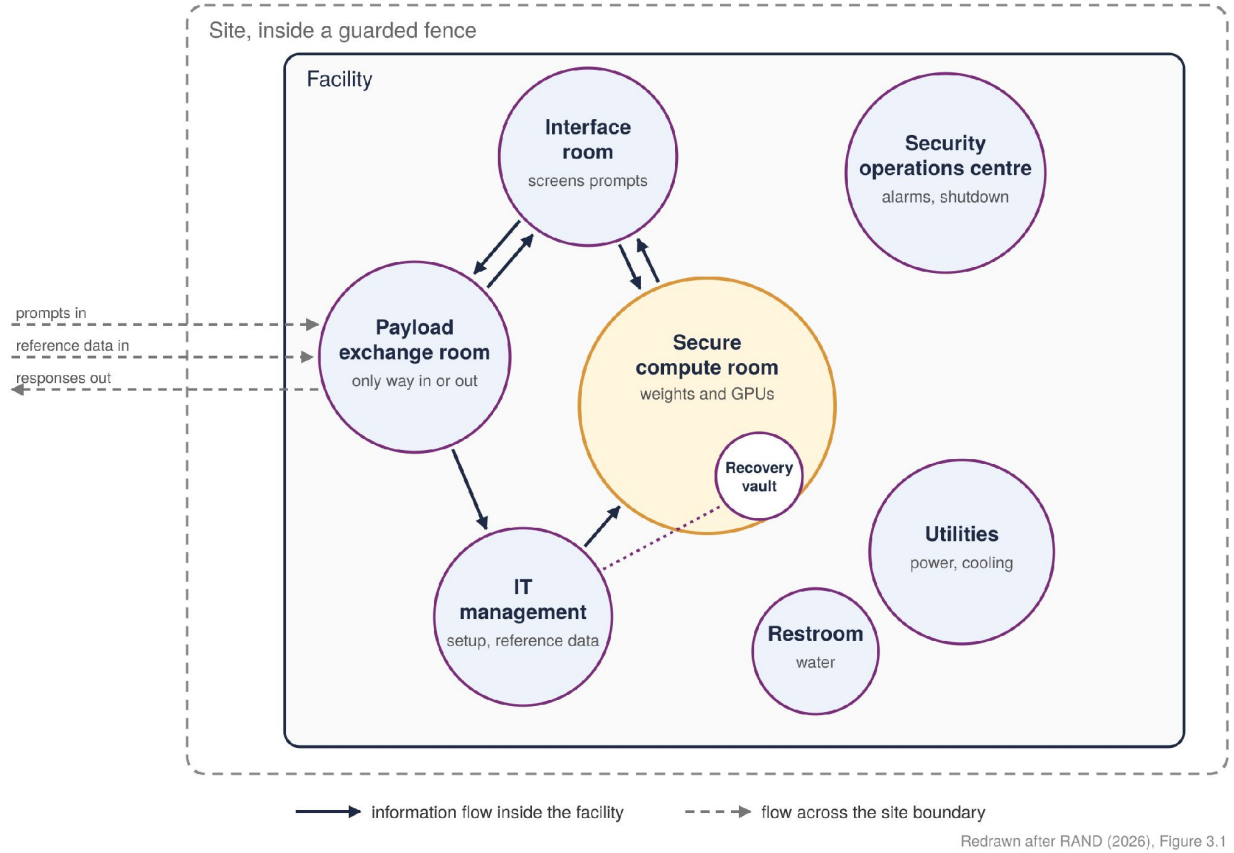
SIDCs are cyber-physical systems

What is an SIDC? (RAND, 2026)

- A purpose-built data centre for running trained frontier AI models
- Protects model weights, inference algorithms, prompts and responses
- Must resist the most capable nation-state attackers: network, supply chain and front door
- Eight physically isolated rooms (right), with limits on information, access, matter and energy

How it is designed today

- STPA-Sec analysis defines unacceptable losses to hazards, constraints and controls
- Controls are written into system models
- System models are then fed through a complex model-based engineering pipeline



Redrawn after RAND (2026), Figure 3.1

Today's assurance chain vs rapid prototyping

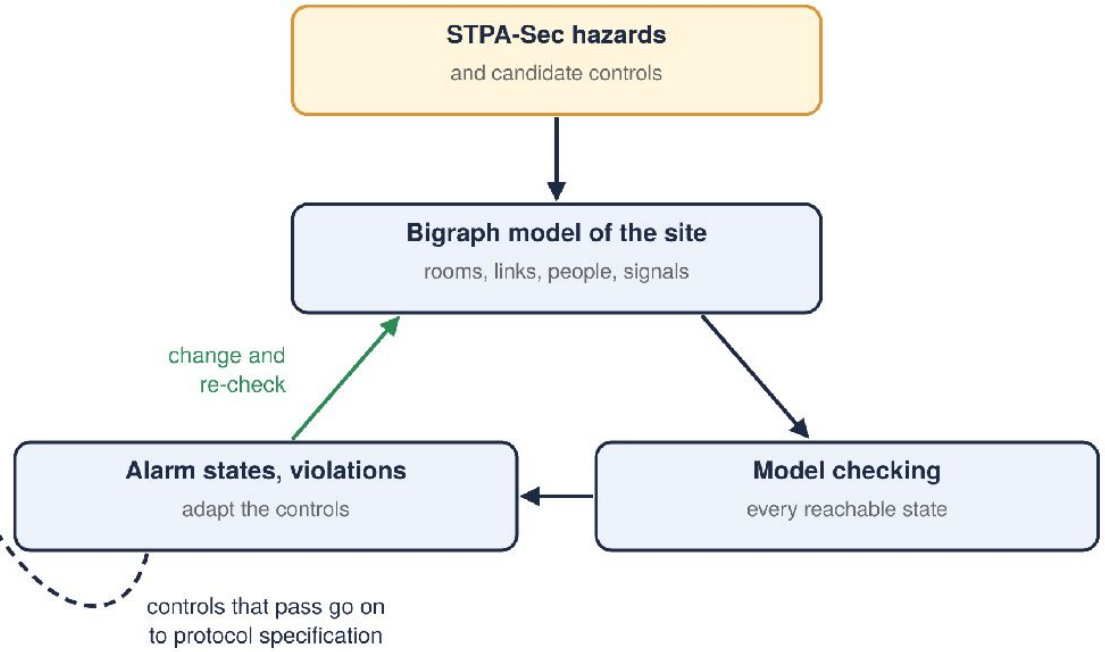
Today: RAND's concept-to-circuit chain

One feature at a time, each layer with its own tool

- **STPA-Sec: losses, hazards, constraints**
- **System model** — SysML
- **Protocol specification** — finite-state protocol
- **Cryptographic analysis** — Tamarin, CryptoVerif
- **State-machine model checking** — TLA+
- **Hardware design** — Bluespec SystemVerilog
- **Circuit-level assertions** — SystemVerilog Assertions
- **Test and red team**

Rapid prototyping: this work

The whole site in one model, checked early



RAND: a system model organises requirements, but it does not prove the system is secure (p. 34).
Each tool's claims stop at its own layer, for example the protocol checker does not cover the architecture around it (p. 34).

Approach: from a spec to a model checked transition system




```
# Controls: kinds of node and their ports
ctrl Outside = 0; ctrl Site = 0; ctrl Utilities = 0;
ctrl Exchange = 1; ctrl Interface = 2; ctrl Compute = 1;  # diode ends
ctrl Request = 0; atomic ctrl Screened = 0; atomic ctrl Response = 0;
atomic ctrl GPU = 1; ctrl Feed = 2; ctrl Sensor = 1; atomic ctrl Trace = 0;

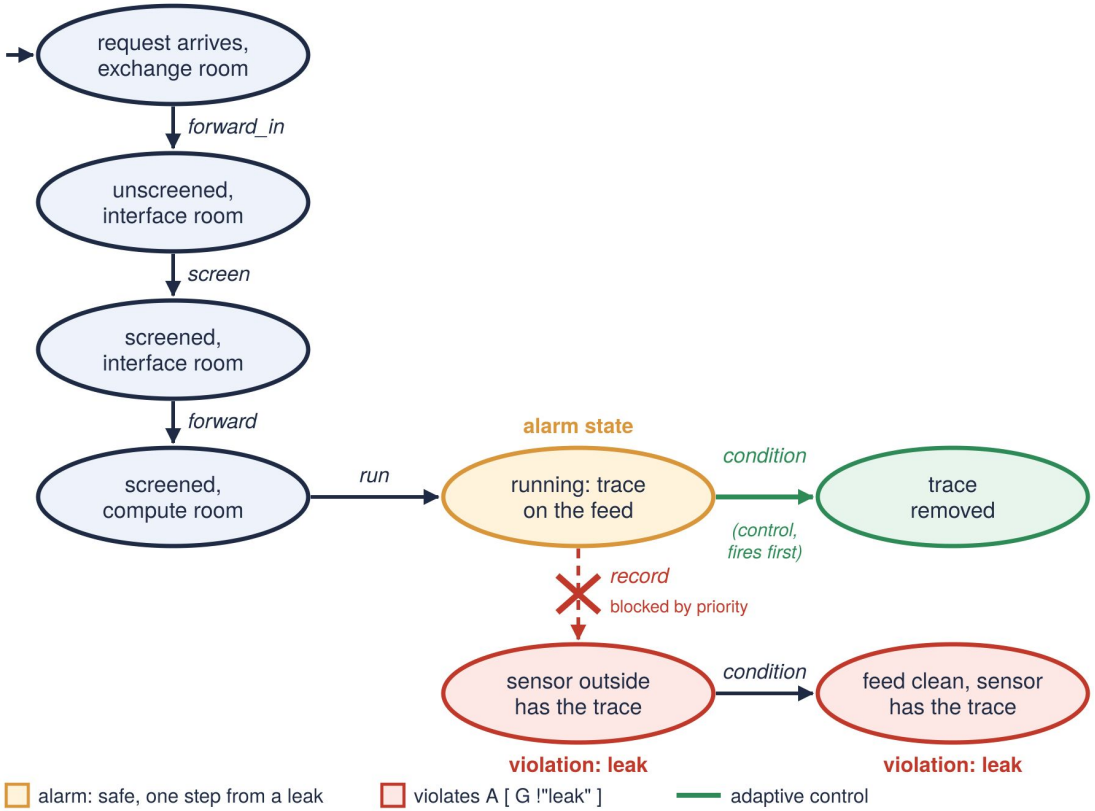
# An inference request has just arrived in the initial state
big sidc = /d1 /d2 /p /g ( Outside.Sensor{g}
  | Site.( Exchange{d1}.Request | Interface{d1,d2}
         | Compute{d2}.GPU{p} | Utilities.Feed{p,g} ) );

# The request's path: each diode is a one-way rule
react forward_in = Exchange{d1}.(Request | id) || Interface{d1,d2}.id
  --> Exchange{d1}.id || Interface{d1,d2}.(Request | id);
react screen = Interface{d1,d2}.(Request | id)
  --> Interface{d1,d2}.(Request.Screened | id);
react forward = Interface{d1,d2}.(Request.Screened | id) || Compute{d2}.id
  --> Interface{d1,d2}.id || Compute{d2}.(Request.Screened | id);

# The power side channel
react run = Compute{d2}.(Request.Screened | GPU{p} | id) || Feed{p,g}
  --> Compute{d2}.(Response | GPU{p} | id) || Feed{p,g}.Trace;
react condition = Feed{p,g}.Trace --> Feed{p,g};
react record = Feed{p,g}.Trace || Sensor{g}
  --> Feed{p,g}.Trace || Sensor{g}.Trace;

# Violation: a trace has leaked off the site to the attacker owned sensor
big leak = Sensor{g}.Trace;

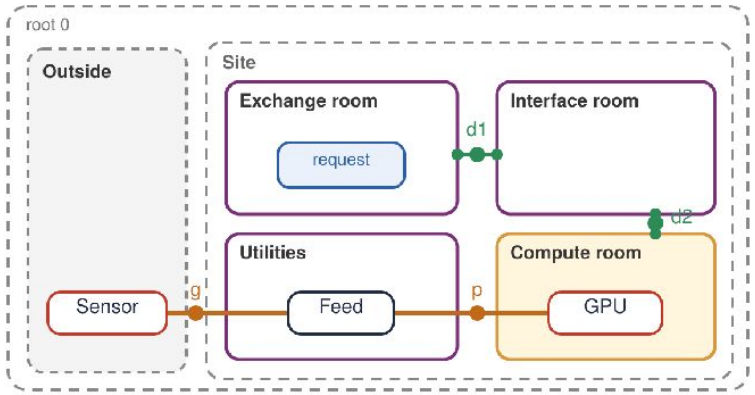
begin brs
  init sidc;
  rules = [ {condition},     # Adaptive control: fires first
            {forward_in, screen, forward, run, record} ];
  preds = { leak };
end
```



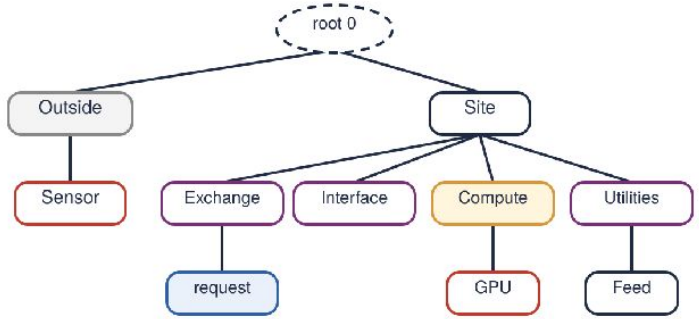
Without priority: 8 states, the leak is reachable. With condition first: 6 states, no leak. Derived by hand, not yet run.

Tools: BigraphER, PRISM and mCRL2 in one Nix shell, github.com/bredamatt/bignix

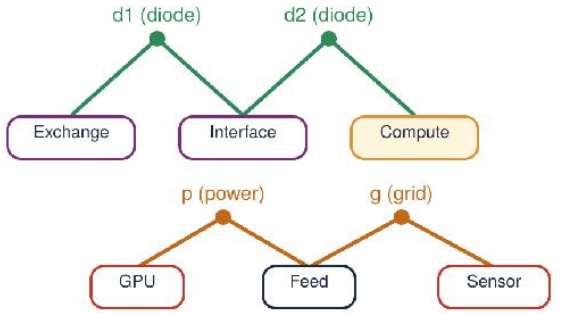
Bigraphs and rewriting example: an inference request



The bigraph

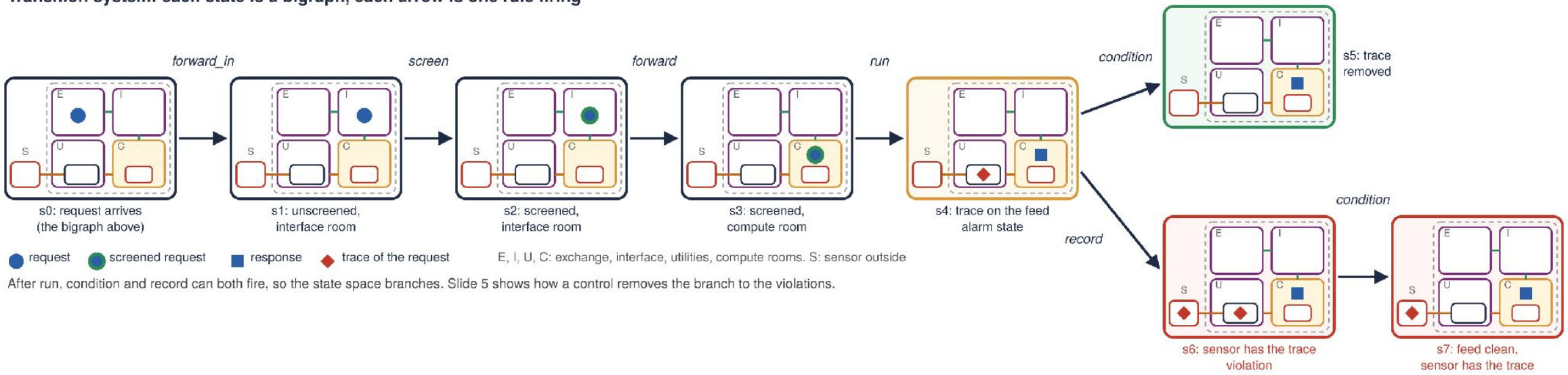


Place graph: what is inside what



Link graph: what is connected to what

Transition system: each state is a bigraph, each arrow is one rule firing



After run, condition and record can both fire, so the state space branches. Slide 5 shows how a control removes the branch to the violations.

What remains

Model

- Extend the request model to the full SIDC layout
- Model emission fields as overlapping regions, using bigraphs with sharing

Calibrate

- Add probabilities for detection and leakage, giving Markov chains, or Markov decision processes
- Calibrate the model against realistic side-channel attacks in data centres

Check

- Run the model in BigraphER and check properties drawn from RAND's hazards
- Find alarm states and derive adaptive controls that prevent, circumvent, or mitigate issues

Stretch goals

- Consider internal threats
- Define lower-level details (rack topology, wiring, etc.)
- Consider AI agents